ML-I LAB PROJECT | P46

P46

Mushroom: Edible or Poisonous?

A Machine Learning Classification Project using a Decision Tree

Student	Jaydeep Rathod
Roll Number	D51
Enrollment Number	24004501210361
Division	E
Project ID	P46
Course	ML-I Lab Project

This report describes the dataset, data cleaning, exploratory analysis, Decision Tree training, model evaluation and final observations.

1. Project Overview

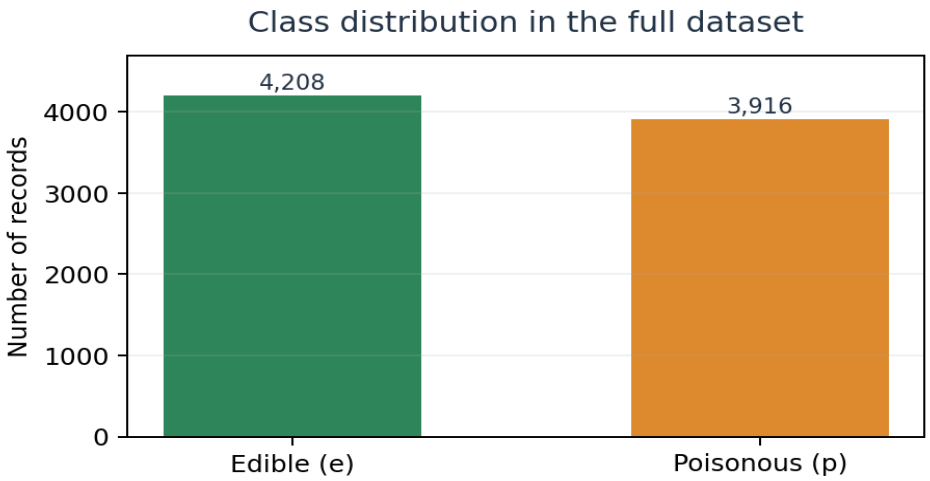
Problem statement. The goal of this project is to classify a mushroom as edible (e) or poisonous (p) using its recorded features, such as cap shape, odor, gill size, stalk characteristics, spore-print color and habitat. The target column is **class**. This is a binary classification problem.

Objectives. Load and clean a real dataset with Python; understand the feature values; create charts; convert categorical data into a model-ready format; train a Decision Tree; compare settings; and evaluate the model with more than accuracy alone.

Dataset summary

Item	Value
Dataset source	Kaggle - Mushroom Classification dataset
Original file	mushrooms.csv
Original size	8,124 rows and 23 columns
Target column	class: e = edible, p = poisonous
Original input features	22 categorical features (before dropping the constant veil-type feature)
Model used	Decision Tree Classifier
Main evaluation focus	Recall for the poisonous class

Class distribution



The dataset contains 4,208 edible mushrooms (51.8%) and 3,916 poisonous mushrooms (48.2%). The two groups are fairly balanced.

2. Data Dictionary

The original CSV columns are categorical values. The feature names below describe the information represented by each column.

Column name	Meaning	Type	Keep / drop
class	Target label: edible (e) or poisonous (p)	Categorical	Target
cap-shape	Shape of the mushroom cap	Categorical	Input feature
cap-surface	Surface texture of the cap	Categorical	Input feature
cap-color	Color of the cap	Categorical	Input feature
bruises	Whether bruising is recorded	Categorical	Input feature
odor	Mushroom odor category	Categorical	Input feature
gill-attachment	How the gills attach to the stalk	Categorical	Input feature
gill-spacing	Spacing between the gills	Categorical	Input feature
gill-size	Size category of the gills	Categorical	Input feature
gill-color	Color category of the gills	Categorical	Input feature
stalk-shape	Shape of the stalk	Categorical	Input feature
stalk-root	Type of stalk root; contained ? markers in the source file	Categorical	Input feature
stalk-surface-above-ring	Stalk surface above the ring	Categorical	Input feature
stalk-surface-below-ring	Stalk surface below the ring	Categorical	Input feature
stalk-color-above-ring	Stalk color above the ring	Categorical	Input feature
stalk-color-below-ring	Stalk color below the ring	Categorical	Input feature
veil-type	Veil type; one value in all rows, so removed during cleaning	Categorical	Dropped
veil-color	Color of the veil	Categorical	Input feature
ring-number	Number of rings	Categorical	Input feature
ring-type	Ring type category	Categorical	Input feature
spore-print-color	Color category of the spore print	Categorical	Input feature
population	Population category in the dataset	Categorical	Input feature
habitat	Habitat category in the dataset	Categorical	Input feature
odor_presence	New feature derived from odor: no_odor or odor_present	Categorical (new)	Derived feature

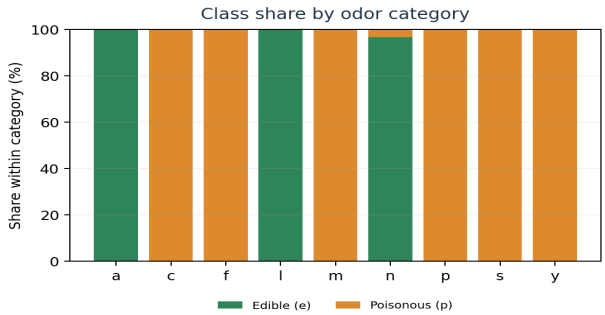
Note: The categorical values are short codes, rather than numeric measurements. The target labels were encoded as e = 0 and p = 1 by LabelEncoder for model training.

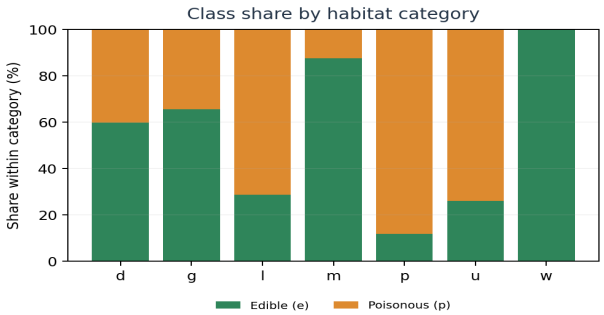
3. Data Cleaning and Exploratory Analysis

The data was cleaned using Python and Pandas. Excel was not used to modify the dataset.

Action	Result	Reason
Check duplicates	0 duplicate rows removed	No exact duplicate records were found.
Handle ? values	2,480 values in stalk-root converted to missing values	The ? symbol was text, not a Pandas missing value.
Fill missing features	Missing values filled with the most common category (mode)	All feature columns were categorical.
Drop veil-type	Column removed; it had one unique value	A constant column provides no useful variation.
Final cleaned data	8,124 rows and 22 columns after cleaning	No missing values remained.

EDA charts and observations





Five observations from the dataset

1. The two classes are fairly balanced: 4,208 edible and 3,916 poisonous records.

2. Odor categories a and l are 100% edible in this dataset; categories c, f, m, p, s and y are 100% poisonous.

3. Odor category n is 96.6% edible, while the remaining 3.4% is poisonous.

4. Gill-size category n is 88.5% poisonous; category b is 69.9% edible.

5. Spore-print category h is 97.1% poisonous, and habitat category p is 88.1% poisonous. Habitat w is 100% edible in this dataset.

Safety note: These patterns describe this dataset only. The model must not be used to decide whether a wild mushroom is safe to eat.

4. Model Building and Comparison

A new feature, **odor_presence**, was created by grouping odor values into no_odor and odor_present. One-hot encoding converted the categorical input features into binary columns. The encoded dataset had 95 input columns before feature selection.

The data was split into 80% training records (6,499) and 20% testing records (1,625), using stratify=y and random_state=42. SelectKBest with the ANOVA F-test selected 15 features and was fitted on the training data only.

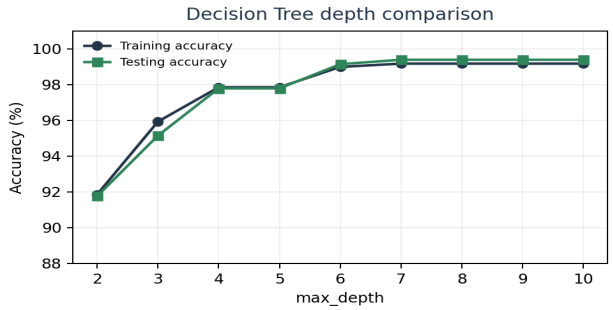
Selected features

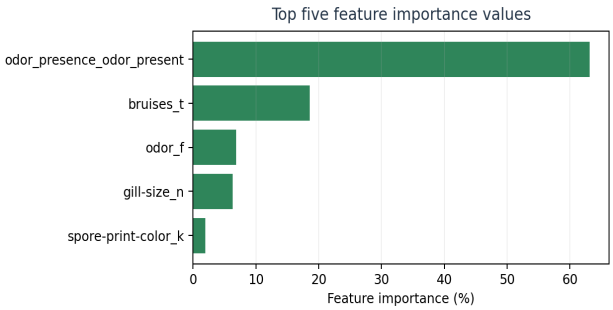
bruises_t, odor_f, odor_n, gill-size_n, stalk-surface-above-ring_k, stalk-surface-above-ring_s, stalk-surface-below-ring_k, stalk-surface-below-ring_s, ring-type_l, ring-type_p, spore-print-color_h, spore-print-color_k, spore-print-color_n, population_v, odor_presence_odor_present

Decision Tree settings tested

Setting	Accuracy	Precision	Recall	F1	Specificity	AUC
Baseline	99.38%	98.74%	100.00%	99.37%	98.81%	0.9999
criterion=gini	99.38%	98.74%	100.00%	99.37%	98.81%	0.9999
criterion=entropy	99.38%	98.74%	100.00%	99.37%	98.81%	0.9999
max_depth=7	99.38%	98.74%	100.00%	99.37%	98.81%	0.9999

The tested settings were the baseline tree, criterion=gini, criterion=entropy and max_depth=7. In this run, the four settings produced the same test-set metrics after rounding, including 100% poisonous-class recall. Since the key metric tied, the baseline Decision Tree was retained as the final model. The first depth that achieved the highest test accuracy was 7.



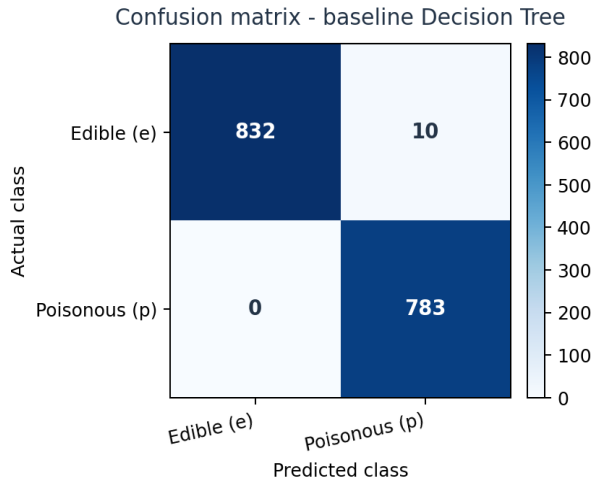


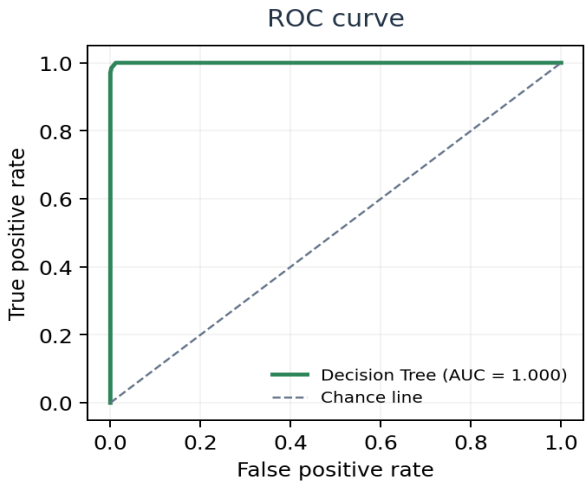
The derived odor_presence feature had the highest importance in the depth-limited tree, followed by bruises_t and odor_f. Since odor_presence is derived from odor, these features contain related information.

5. Final Evaluation and Conclusion

99.38%	98.74%	100.0%	99.37%
Accuracy	Poisonous precision	Poisonous recall	F1-score

Specificity: 98.81% | ROC-AUC: 0.9999





The confusion matrix shows 832 edible mushrooms correctly predicted as edible, 10 edible mushrooms incorrectly predicted as poisonous, 783 poisonous mushrooms correctly predicted as poisonous, and 0 poisonous mushrooms incorrectly predicted as edible.

Conclusion

In this project, a Decision Tree model was used to classify mushrooms as edible or poisonous. The dataset was cleaned using Python, missing values were handled, categorical features were encoded, and 15 features were selected for training. The baseline model achieved 99.38% accuracy and 100% recall for the poisonous class on the test split. The results show that the selected features were useful for classifying records in this dataset. The project helped demonstrate data cleaning, EDA, train-test splitting, feature selection, model comparison and evaluation metrics.

Limitations and responsible use

The result is based on one train-test split of a public dataset. A high score does not prove that the model is safe for real mushrooms found outdoors. Do not eat a mushroom based on this model prediction; real-world identification requires qualified expertise.

Sources

1. Faculty-provided ML-I Lab Project Guide, P46 - Mushroom: Edible or Poisonous? (project workflow and submission requirements).

2. Kaggle, Mushroom Classification dataset: https://www.kaggle.com/datasets/uciml/mushroom-classification

3. Analysis reproduced from the supplied mushrooms.csv file using Pandas, scikit-learn, Matplotlib and Seaborn-style categorical plots.